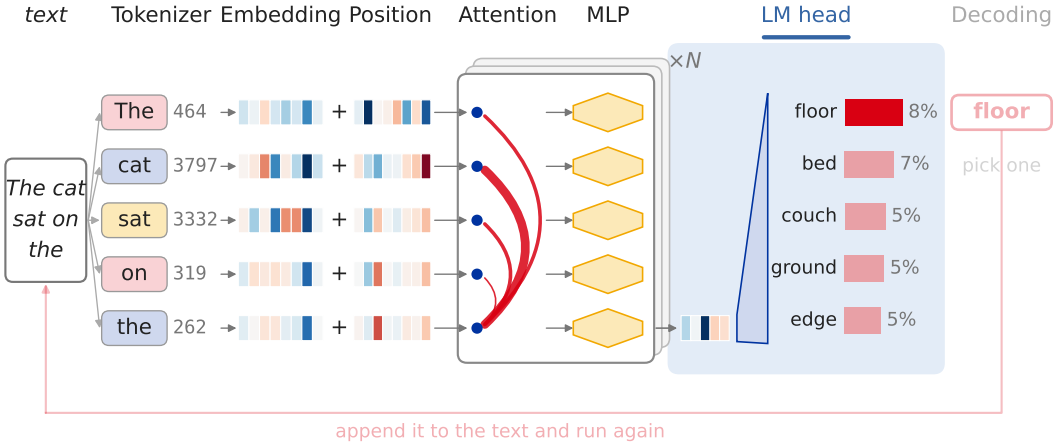
Transformers in the World

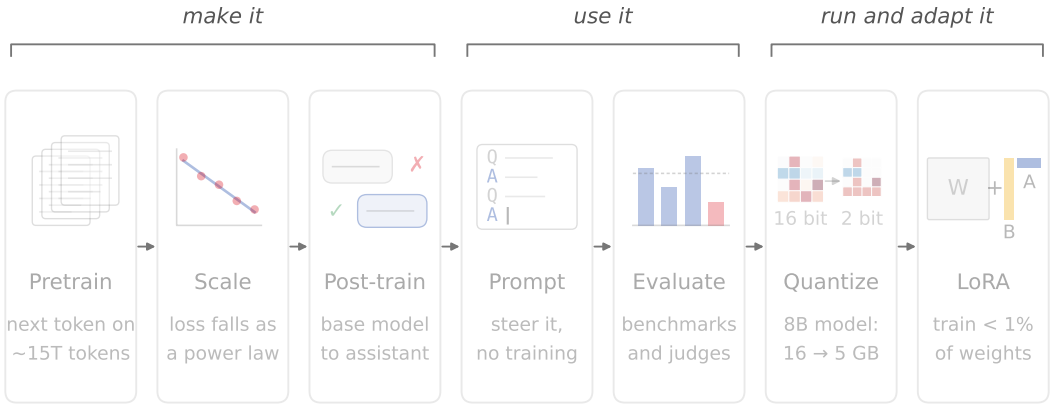
The family, how it learns, and what it is still bad at

Where we are: one forward pass



Today: the **LM head** - from the last vector to a distribution over the vocabulary - and the three ways the same block gets wired.

Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

The same block, three different jobs

You can now build a Transformer block. But the block alone does not tell you whether you are building **BERT**, **GPT** or a translation system.

The difference is not the block. It is one decision:

who is allowed to look at whom?

Change the attention mask and the same architecture becomes a text classifier, a chatbot, or a translator. That is this lecture's first section - and then we ask why this design beat everything else, and where it is still weak.

Outline

The family

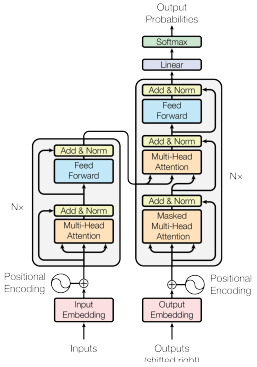
How it learns

Why it won, and what it costs

The family

One block, three ways of wiring it.

The original: an encoder and a decoder



Vaswani et al. (2017), Figure 1

The 2017 paper was about **translation**, so it has two stacks:

Encoder – reads the source sentence. Every token may see every other token.

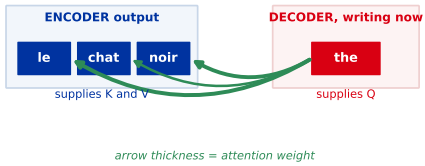
Decoder – writes the target sentence, one token at a time, so it is **causally masked** (LLM-4).

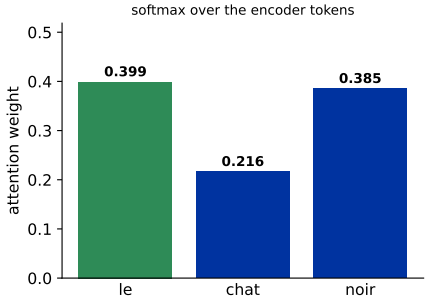
They are joined by **cross-attention**: the decoder supplies the queries, and the *encoder's* output supplies the keys and values.

Self-attention asks "what in *this* sentence matters to me?" Cross-attention asks "what in *that other* sequence matters to me?" Identical mechanism; the keys and values just come from somewhere else.

Cross-attention, with actual numbers

The decoder is writing an English sentence and has produced “the”. The encoder has already read the French source. Where should “the” look?





One head, $d_k = 4$. Scores $+0.339$, -0.273, $+0.304$ become weights **0.399**, **0.216**, **0.385** after softmax – and they sum to 1, exactly as in LLM-3.

What actually changed

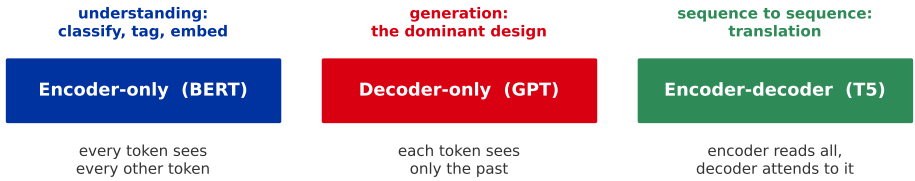
Compare with the self-attention you computed in LLM-3. The formula is character-for-character the same:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

	Self-attention	**Cross-attention**
Q comes from	this sequence	**the decoder**
K, V come from	this sequence	**the encoder**

That is the entire difference: **two of the three inputs come from somewhere else**. No new mechanism, no new maths – which is why the original paper could reuse the same block for both.

Three architectures



same block, three ways of deciding who may look at whom

schematic

Nobody changed the block. **Encoder-only** sees in both directions and is good at understanding. **Decoder-only** sees only backwards, which makes it able to generate. **Encoder-decoder** keeps both and joins them with cross-attention.

Why decoder-only won the last five years

- **One objective, unlimited data.** Predicting the next token needs no labels, so any text on the internet is training data.
- **It subsumes the others.** "Translate this: ..." is just a prompt. Tasks that needed a separate architecture became a formatting question.
- **It is the simplest thing to scale.** One stack, one loss, no encoder to keep in sync.

This is a statement about the last few years, not a theorem. Encoder-only models are still the right tool when you want an *embedding* rather than a continuation – which is exactly what the retrieval chapter used them for.

How it learns

One objective, and the trick that turns it into a chatbot.

The objective you have already seen

In **L21** we factorised a language model:

$$P(y_1, \ldots, y_T) = \prod_{t=1}^{T} P(y_t \mid y_{<t})$$

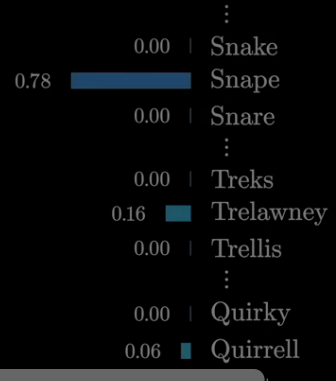
That is all GPT is trained to do. Show it a prefix, ask for the next token, compare with the truth, backpropagate. The loss is plain cross-entropy.

Nothing about the objective is new. What changed is that the **causal mask** (LLM-4) lets every position in a sequence be a training example in the same forward pass, and that the architecture is parallel enough to make internet-scale data affordable.

So the modern language model is an old idea with a new engine.

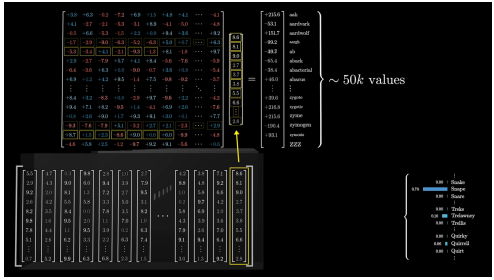
Harry Potter was a highly unusual boy ... least favourite teacher, Professor ???





Next-token prediction, in practice: the model outputs a distribution over the whole vocabulary, and one continuation is far more likely than the rest.

From a vector to a word



3Blue1Brown

After the last block, the final token's vector is multiplied by an **unembedding matrix** – one row per vocabulary entry, so roughly 50,000 numbers come out.

Those are **logits**. A softmax turns them into a probability distribution, and you sample from it.

That is the entire output stage.

The model does not choose a word. It produces a *distribution* over every word, and the choosing is a separate decision – which is where temperature and the other decoding strategies live, later in this chapter.

Autocomplete is not a chatbot

A model trained this way continues text. Given "the capital of Armenia is", it says "Yerevan". Given "how do I reset my password?", it might continue with *another question*, because that is what web pages look like.

The fix that started the current era is embarrassingly simple: put the conversation in a format where the helpful continuation *is* the likely one.

```
You are a helpful assistant.
User:  how do I reset my password?
Assistant:
```

Now the most likely continuation of the *text* is the answer you wanted. The system prompt is not a new capability - it is a **framing** that makes autocomplete behave like a conversation. Fine-tuning, and then training on human preferences, make it reliable - that is the `llm_training` track.

Why it won, and what it costs

Every architecture buys something with something.

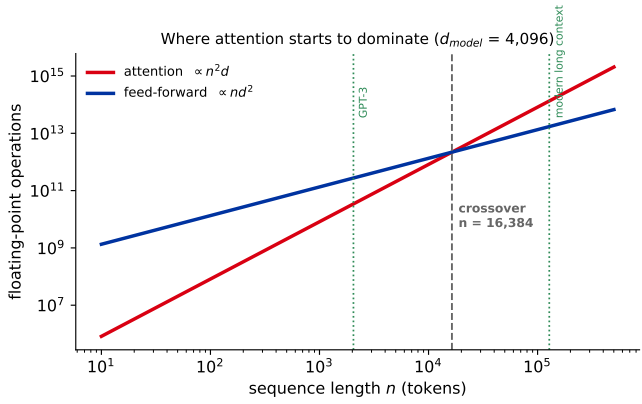
Why transformers won

1. **It parallelises.** An RNN had to walk the sequence in order. Attention is three big matmuls over the whole sequence at once (LLM-3), which is exactly the workload a GPU wants.
2. **Because it parallelises, it scales.** You can afford more data and more parameters.
3. **Because it scales, it kept getting better** – and past certain sizes, qualitatively so. The clearest example is **in-context learning**: give a model a few examples in the prompt and it follows the pattern, *with no weight updates at all*. Small models essentially cannot do this; past a certain scale they can.

Notice the shape of that argument: the win is not that attention is a cleverer model of language. It is that attention is a **better fit for the hardware**, and everything else followed from being able to train bigger. Architecture that suits the machine beats architecture that suits your intuition.

The bill: attention is quadratic

Every token attends to every token, so the score matrix is $n \times n$. Double the context and you quadruple that cost.



Computed for $d_{\text{model}} = 4096$ with a $4\times$ feed-forward.

Read that graph carefully

The usual story is "attention is the bottleneck". The arithmetic says: **not until the sequence gets long**.

You can work out where it flips in two lines. Attention costs about $n^2 d$; the feed-forward costs about $nd^2 f$, where f is the widening factor (usually 4). Set them equal:

$$n^2 d = nd^2 f \quad \Longrightarrow \quad \boxed{n = d \cdot f}$$

So for $d_{\text{model}} = 4096$ and $f = 4$, the crossover is at $4096 \times 4 = \mathbf{16{,}384}$ tokens. Halve the model and you halve the crossover.

So for a 2,048-token context, optimising attention is optimising the smaller half. "Quadratic" describes how the cost *grows*, not where the cost *is* at every size – worth knowing before you optimise the wrong thing. FlashAttention and the sparse variants attack the n^2 term, later in this chapter.

The honest limit: almost no inductive bias

Every architecture in this course encoded an assumption about its data:

Model	**Built-in assumption**	**Consequence**
Decision tree (ch4)	axis-aligned splits matter	works on tiny tabular data
CNN (ch6)	nearby pixels are related	learns vision from thousands of images
RNN (ch7)	order matters, recent matters more	handles sequences by construction
Transformer	**almost nothing**	**needs enormous data**

A Transformer does not know that neighbouring words are related, or even that words have an order – we had to *bolt position on* in LLM-4.

That weakness is the same fact as its strength. Assuming nothing means it can learn any structure, and **must** learn all of it from data. This is why a ViT loses to a CNN on a small image dataset and wins on a huge one, and why transformers arrived only once the data and the compute did.

Same block, everywhere

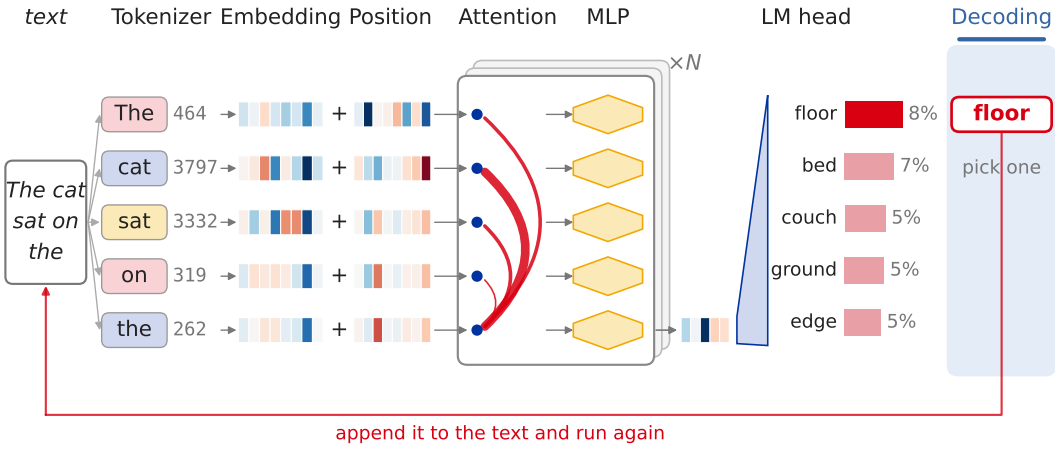
Domain	What became a token	Chapter
Text	subword pieces	this one, and `llm_training`
Images (ViT)	16×16 patches	ch12, vision-language models
Audio	spectrogram frames or codec codes	ch13
Proteins	amino acids	AlphaFold
Actions	robot commands	ch15, vision-language-action

Nobody redesigned the block for any of these. They redesigned what counts as a **token**. That is the practical legacy of the architecture: if you can serialise your problem into a sequence of vectors, you can throw a Transformer at it.

Recap

- **One block, three wirings.** Encoder-only (BERT) understands, decoder-only (GPT) generates, encoder-decoder (T5) translates. The difference is the mask, not the block.
- **Cross-attention** is ordinary attention with the keys and values taken from another sequence.
- **Training is L21's objective** – predict the next token – made affordable by the causal mask and by hardware-friendly parallelism.
- **The output is a distribution**, not a word. A system prompt turns autocomplete into a chatbot by making the helpful continuation the likely one.
- **It won because it fits the GPU**, not because it is a better theory of language.
- **It is quadratic in sequence length**, but at $d_{\text{model}} = 4096$ that only dominates past 16,384 tokens.
- **It assumes almost nothing**, so it can learn anything and needs vast data to do it.

Next on the map



Next: **decoding** - how a distribution over the vocabulary becomes the text you read. That finishes the forward pass; after it, where the weights come from.

That is the T in GPT.

You have built the block, wired the family,
and priced what it costs.

Next in this chapter: decoding,
then how these models are actually trained at scale.